

Grouping, Categorization, and Upload Implementation

This document explains how CSV/XLS/XLSX uploads, transaction grouping, and categorization are implemented in FinSight Advisor. It only covers files that participate in these operations.

High-Level Flow

The main operational flow is:

1. The frontend sends an authenticated file upload request with a selected `dept_id`.
2. The backend reads the CSV/XLS/XLSX file into a Pandas dataframe.
3. Upload columns are normalized into the application's canonical transaction fields.
4. Debit/credit columns are resolved into `amount` plus `transaction_type`.
5. Each row is validated, converted into a `Transaction`, and linked to an `UploadBatch`.
6. Duplicate rows are skipped using a deterministic `dedupe_hash`.
7. Grouping embeds transaction text and assigns transactions to semantic groups.
8. Categorization can do two different things:
 - assign each transaction a simple `necessary`, `unnecessary`, or `uncategorized` label;
 - suggest expense categories for groups using Gemini, then let an admin approve or reject them.

Upload Operation: CSV/XLS/XLSX to Database

Frontend Request

The upload starts from the admin dashboard.

Relevant files:

- `frontend/pages/AdminDashboard.tsx`
- `frontend/lib/api.ts`
- `frontend/types.ts`

In `frontend/pages/AdminDashboard.tsx`, `handleUpload` is used for the main transaction upload workflow. It checks that a department and file are selected, then calls:

```
await api.uploadTransactions(selectedDeptId, file);
```

There is also `handleForensicUpload`, which uses the same backend upload endpoint but is presented in the forensic workflow.

In `frontend/lib/api.ts`, `uploadTransactions` builds a `FormData` payload:

```
const form = new FormData();
form.append('dept_id', deptId);
form.append('file', file);
return request('/transactions/upload', { method: 'POST', body: form });
```

The shared `request` helper also attaches the bearer token from `localStorage`, so the backend upload endpoint is authenticated.

`frontend/types.ts` defines the frontend transaction and upload-related shapes that the dashboard expects after reloads, including `Transaction`, `UploadBatchSummary`, `ExpenseCategory`, and `ExpenseGroupSummary`.

Backend Upload Endpoint

Main file:

- `backend/app/transaction/router.py`

The upload endpoint is:

```
@router.post("/upload", response_model=UploadResponse)
async def upload_transactions(...)
```

It receives:

- `file`: the uploaded CSV/XLS/XLSX file;
- `dept_id`: the target department id;
- `current_user`: injected by authentication;
- `db`: SQLAlchemy database session.

The endpoint first checks that the department exists:

```
department = db.query(Department).filter(Department.department_id == dept_id).first()
```

If no department is found, it returns `404`.

Then it reads and normalizes the file:

```
contents = await file.read()
dataframe = read_uploaded_file(file.filename or "upload.csv", contents)
dataframe = normalize_upload_columns(dataframe)
dataframe = resolve_amount_and_type(dataframe)
ensure_dataframe_columns(dataframe, REQUIRED_COLUMNS)
```

The required canonical columns are:

```
REQUIRED_COLUMNS = ["transaction_date", "description", "chart_acc_head"]
```

The upload must also contain either `amount` or `credit` before amount resolution succeeds.

File Reading and Amount Resolution

Main file:

- `backend/app/services/dataframe.py`

`read_uploaded_file` chooses a parser by file extension:

- `.csv` uses `pd.read_csv`;
- `.xls` and `.xlsx` use `pd.read_excel`;
- any other extension raises `ValueError`.

After reading, it calls `normalize_upload_dataframe`, which lowercases column names and maps known aliases into canonical names. Examples:

- `date` or `txn_date` -> `transaction_date`
- `debit`, `debit_amount`, `transaction_amount` -> `amount`
- `credit_amount` -> `credit`
- `narration`, `details`, `remarks` -> `description`
- `account_head`, `chart_account_head` -> `chart_acc_head`
- `voucher number` -> `voucher_number`

`resolve_amount_and_type` handles ledger files that have separate debit and credit columns. It creates:

- `amount`: absolute numeric value from the debit side unless the row is credit-only;
- `transaction_type`: `debit` or `credit`.

This is important because downstream budget and forensic code needs to know whether a row is money-out or money-in.

Additional Upload Column Normalization

Main file:

- `backend/app/transaction/router.py`

After `read_uploaded_file`, the upload endpoint also calls `normalize_upload_columns`. This second normalization layer handles additional aliases seen in accounting exports, for example:

- `Narration` -> `description`
- `Chart of Account Head` -> `chart_acc_head`
- `Group No` -> `group_no`
- `Account Head Group` -> `account_head_group`
- `Voucher_Type` -> `voucher_type`
- `Reference Number` -> `po_number`

This means the upload flow supports both normalized lowercase exports and title-case Excel-style ledger exports.

Shared Upload Helpers

Main file:

- `backend/app/services/common.py`

The upload endpoint uses:

- `ensure_dataframe_columns`: verifies required columns are present.
- `clean_chart_account_head`: lowercases account heads, removes bracketed text, numbers, and unsupported punctuation, then collapses whitespace.
- `compute_dedupe_hash`: creates a SHA-256 hash from department id, transaction date, amount, description, invoice id, and PO number.
- `normalize_bool`: converts values like `true`, `yes`, `1`, and `y` into booleans.

The dedupe hash prevents repeated uploads of the same transaction row for the same department.

UploadBatch Creation

Main files:

- `backend/app/transaction/router.py`
- `backend/app/models.py`
- `backend/schema.sql`

Before inserting transaction rows, the backend creates an `UploadBatch`:

```
batch = UploadBatch(  
    department_id=dept_id,  
    source_file_name=file.filename or "upload.csv",  
    uploaded_by=current_user.user_id,  
    row_count=len(dataframe.index),  
    status="processing",  
)
```

The `UploadBatch` table stores metadata about the uploaded file:

- `upload_batch_id`
- `department_id`
- `source_file_name`
- `uploaded_by`
- `uploaded_at`
- `row_count`
- `status`

Every inserted transaction receives the batch id in `upload_batch_id`, so later features can trace transactions back to the uploaded file.

Duplicate Detection

Main file:

- `backend/app/transaction/router.py`

Before processing rows, the endpoint loads all existing hashes for the department:

```

existing_hashes = {
    dedupe_hash
    for (dedupe_hash,) in db.query(Transaction.dedupe_hash)
    .filter(
        Transaction.department_id == dept_id,
        Transaction.dedupe_hash.isnot(None),
    )
    .all()
}

```

This avoids querying the database once per row, which is especially important when the database is remote, such as Supabase.

For each row, the backend computes a new hash with `compute_dedupe_hash`. If the hash already exists, the row is counted as a duplicate and skipped.

Row-by-Row Transaction Insert

Main files:

- `backend/app/transaction/router.py`
- `backend/app/models.py`
- `backend/schema.sql`

For each dataframe row, the upload endpoint:

1. parses `transaction_date` with Pandas;
2. converts `amount` to `float`;
3. normalizes `transaction_type`;
4. extracts optional text fields;
5. cleans `chart_acc_head`;
6. computes `dedupe_hash`;
7. creates a `Transaction` ORM object;
8. adds it to the SQLAlchemy session.

The inserted `Transaction` stores fields including:

- `transaction_id`
- `department_id`
- `transaction_date`
- `amount`
- `transaction_type`
- `description`

- `category`
- `chart_acc_head`
- `cleaned_chart_acc_head`
- `group_no`
- `group_name`
- `payment_method`
- `invoice_id`
- `voucher_number`
- `account_head_group`
- `voucher_type`
- `po_number`
- `has_receipt`
- `approval_status`
- `source_file_name`
- `upload_batch_id`
- `dedupe_hash`

If a row fails parsing or conversion, the endpoint does not stop the whole upload. It increments `rows_failed` and records the row number and error message in `failed_rows`.

At the end:

- `batch.status` becomes `completed` if all rows succeeded;
- `batch.status` becomes `completed_with_errors` if any row failed;
- the transaction rows and upload batch update are committed together.

The API returns:

```
{
  "success": rows_failed == 0,
  "upload_batch_id": batch.upload_batch_id,
  "rows_processed": rows_processed,
  "rows_failed": rows_failed,
  "duplicate_rows": duplicate_rows,
  "failed_rows": failed_rows[:50],
}
```

Grouping Operation

Frontend Trigger

Relevant files:

- `frontend/pages/AdminDashboard.tsx`
- `frontend/lib/api.ts`

In `AdminDashboard.tsx`, grouping is triggered through:

```
triggerAction('group')
```

That calls:

```
await api.runGrouping(selectedDeptId);
```

In `frontend/lib/api.ts`, this sends:

```
request('/grouping/assign-groups', {  
  method: 'POST',  
  body: JSON.stringify({ dept_id: deptId }),  
})
```

Backend Grouping Endpoint

Main file:

- `backend/app/grouping/router.py`

The endpoint is:

```
@router.post("/assign-groups", response_model=AssignGroupsResponse)  
def assign_transaction_groups(...)
```

It accepts:

- `dept_id`
- optional `similarity_threshold`, defaulting to `0.7`

The endpoint:

1. verifies that the department exists;

2. loads all department transactions ordered by date;
3. loads existing `Group` rows for the department;
4. extracts text for each transaction;
5. embeds those texts;
6. compares transaction embeddings to existing group profiles;
7. assigns each transaction to the best matching group if similarity is high enough;
8. creates a new group if no existing group passes the threshold.

Text Used for Grouping

Main file:

- `backend/app/grouping/router.py`

Grouping uses `transaction_group_text`:

```
return (transaction.description or transaction.chart_acc_head or transaction.account_head_group or "").strip()
```

The system prefers the transaction description/narration. If that is missing, it falls back to chart account head, then account head group.

Embeddings and Similarity

Main file:

- `backend/app/grouping/grouping_sbert.py`

`encode_texts` uses `sentence-transformers/all-MiniLM-L6-v2` when `sentence-transformers` is available. It returns normalized NumPy embeddings.

If the model package cannot be loaded, the file provides a deterministic fallback embedding based on SHA-256. That fallback keeps the API functional, but semantic quality is lower than the real SBERT model.

`cosine_similarity` compares two embedding vectors and returns a float similarity score.

Existing Group Profiles

Main file:

- `backend/app/grouping/router.py`

For every existing group, the endpoint rebuilds a group profile from the embeddings of its member transactions:

```
profile = np.mean(np.vstack(member_vectors), axis=0)
```

This means old groups can be upgraded if their stored embedding was created from a less useful field. If a group has no matching member transactions but has `representative_text`, the system embeds that representative text.

Assigning or Creating Groups

Main file:

- `backend/app/grouping/router.py`

For each transaction:

1. `clean_chart_account_head` refreshes `txn.cleaned_chart_acc_head`.
2. The transaction embedding is compared with every group profile.
3. If the best similarity is greater than or equal to `similarity_threshold`, the transaction receives:
 - `group_no`
 - `group_name`
4. If no group matches, a new `Group` row is created.

New groups store:

- `dept_id`
- `chart_acc_head_name`
- `group_no`
- `group_name`
- `representative_text`
- `embedding`

The group key for new semantic groups is generated as:

```
description_group_{group_number}
```

Finally, the endpoint commits the updated transactions and new groups.

Grouping Statistics

Main file:

- `backend/app/grouping/router.py`

GET /grouping/dept/{dept_id}/statistics returns:

- total groups;
- grouped transaction count;
- ungrouped transaction count;
- top groups by transaction count.

The frontend uses this data to show the current grouping state.

Categorization Operation

The system has two categorization flows:

1. transaction-level category prediction: necessary, unnecessary, uncategorized;
2. expense-category suggestion and approval for groups.

Both are implemented in:

- backend/app/categorization/router.py

Transaction-Level Categorization

Frontend Trigger

Relevant files:

- frontend/pages/AdminDashboard.tsx
- frontend/lib/api.ts

In AdminDashboard.tsx, simple categorization is triggered by:

```
triggerAction('categorize')
```

This calls:

```
await api.runCategorization(selectedDeptId);
```

In frontend/lib/api.ts, the request goes to:

```
POST /categorization/predict
```

with:

```
{ "dept_id": "..."} }
```

Backend Logic

Main file:

- backend/app/categorization/router.py

The endpoint is:

```
@router.post("/predict")
def categorize_transactions(...)
```

It:

1. verifies the department exists;
2. loads all transactions for that department;
3. counts frequency by `group_name`, `cleaned_chart_acc_head`, or `ungrouped`;
4. builds text from `description` plus account head;
5. embeds the text using `encode_texts`;
6. applies rule-based categorization.

The rule logic is:

- default prediction is `necessary`;
- if a group appears two times or fewer and amount is positive, mark as `unnecessary`;
- if amount is less than 100 and the group appears more than twice, mark as `necessary`;
- if both description and chart account head are missing, mark as `uncategorized`;
- if frequency is two or fewer and the result is not already `necessary`, force `unnecessary`.

For every transaction, the endpoint updates:

- `category`
- `semantic_confidence`

`semantic_confidence` is derived from the average absolute embedding value and clamped between `0.15` and `0.99`.

The endpoint commits the transaction updates and returns counts for each category.

Categorization Summary

Main file:

- `backend/app/categorization/router.py`

`GET /categorization/dept/{dept_id}/summary` counts transactions by:

- `necessary`
- `unnecessary`
- `uncategorized`

The frontend uses this for department dashboard summaries.

Expense Category Suggestion and Review

This is the richer categorization workflow that classifies groups into accounting expense categories.

Frontend Trigger and Review UI

Relevant files:

- `frontend/pages/AdminDashboard.tsx`
- `frontend/lib/api.ts`
- `frontend/types.ts`

In `AdminDashboard.tsx`, expense categorization is triggered by:

```
triggerAction('expense')
```

That calls:

```
const result = await api.runExpenseCategorization(selectedDeptId);
```

The frontend then stores:

```
setExpenseGroups(result.groups);
```

The expense review table displays each group, its sample transactions, Gemini suggestion, confidence, and approval controls.

Admin actions call:

- `api.approveExpenseGroup(...)`
- `api.rejectExpenseGroup(...)`

The frontend types for this flow are:

- `ExpenseCategory`
- `ExpenseGroupSummary`
- `ExpenseCategorizationRunResponse`

Category Catalog

Main file:

- `backend/app/categorization/router.py`

The system starts with these default expense categories:

- Remuneration
- Transportation Cost
- Fuel
- Maintenance
- Office Supplies
- Software
- Rent
- Utilities
- Training

`ensure_initial_expense_categories` creates these as system categories if they do not already exist.

`scoped_categories` returns active categories available to a department. It includes:

- global system categories;
- department-specific categories;
- company categories if the department belongs to a company.

Ensuring Groups Exist Before Expense Categorization

Main file:

- `backend/app/categorization/router.py`

Expense categorization calls:

```
groups = ensure_groups_from_transactions(db, payload.dept_id)
```

This function makes sure every transaction has a usable group. It:

1. loads department transactions;
2. loads existing `Group` rows;
3. maps groups by `chart_acc_head_name` and by `group_no`;
4. cleans account head text;
5. reuses semantic groups when `group_no` already exists;
6. creates a `Group` if needed;
7. fills missing transaction `group_no`, `group_name`, and `expense_category_id` when possible.

This protects the expense categorization workflow from running on ungrouped uploads.

Building Samples for Gemini

Main file:

- `backend/app/categorization/router.py`

`transaction_samples` takes up to three representative transactions from a group. Each sample can include:

- description;
- chart account head;
- account head group;
- voucher type;
- amount;
- transaction type.

The goal is to give Gemini enough real ledger context to classify the whole group without sending every transaction row.

Calling Gemini

Main file:

- `backend/app/categorization/router.py`

The endpoint for suggestions is:

```
@router.post("/expense/predict")
def suggest_expense_categories(...)
```

It:

1. verifies department;

2. loads scoped categories;
3. ensures groups exist;
4. skips groups already approved;
5. builds candidate group summaries;
6. sends those summaries to Gemini;
7. stores Gemini's suggestion on the `Group` row.

Gemini configuration comes from backend environment variables:

- `GEMINI_API_KEY` or `GOOGLE_API_KEY`
- `GEMINI_MODEL`, defaulting to `gemini-3.5-flash-lite`

`build_gemini_prompt` instructs Gemini to:

- choose an existing category when it fits;
- avoid generic catch-all categories like Other or Miscellaneous;
- propose a concise new category only if existing categories do not fit;
- return JSON only.

`call_gemini_for_expense_categories` sends the request to the Google Generative Language API with a JSON response schema.

Storing Suggestions

Main file:

- `backend/app/categorization/router.py`

For each Gemini assignment, the group is updated with:

- `suggested_category_name`
- `suggested_category_confidence`
- `suggested_category_is_new`
- `suggested_category_reason`
- `suggested_category_source`
- `suggested_category_payload`
- `expense_category_status = "pending_review"`

The suggestion is not directly applied to transactions. It waits for admin approval.

Approving Suggestions

Main file:

- `backend/app/categorization/router.py`

The endpoint is:

```
@router.post("/expense-groups/approve")
def approve_expense_category_group(...)
```

It:

1. finds the group by `group_no` or `chart_acc_head_name` ;
2. chooses a category name from the admin payload or the stored suggestion;
3. looks for an existing category with the same normalized key;
4. creates a new `ExpenseCategory` if needed;
5. sets the group's `expense_category_id` ;
6. marks the group as `approved` ;
7. applies the category id to all transactions in that group.

After approval, both the `Group` and all group member `Transaction` rows point to the approved `ExpenseCategory` .

Rejecting Suggestions

Main file:

- `backend/app/categorization/router.py`

The endpoint is:

```
@router.post("/expense-groups/reject")
def reject_expense_category_group(...)
```

It:

1. finds the group;
2. sets `expense_category_status = "rejected"` ;
3. clears suggestion fields;
4. commits the update.

Rejecting a suggestion does not delete the group or transactions.

Reading Expense Groups

Main file:

- `backend/app/categorization/router.py`

`GET /categorization/dept/{dept_id}/expense-groups` returns serialized group data for the review UI.

`serialize_expense_group` includes:

- group identity;
- representative text;
- transaction count;
- sample transactions;
- approved category;
- suggestion details;
- review status.

Database Models and Tables

`backend/app/models.py`

This file defines the SQLAlchemy ORM models used by the upload, grouping, and categorization operations.

Relevant models:

- `Department` : parent entity for transactions, upload batches, groups, and categories.
- `UploadBatch` : one uploaded file and its processing status.
- `Transaction` : one ledger row after parsing and normalization.
- `Group` : semantic or account-head group assigned to transactions.
- `ExpenseCategory` : approved category catalog entries.
- `User` : used for upload ownership and category creation ownership.

Important relationships:

- `Department.transactions`
- `Department.groups`
- `Department.upload_batches`
- `Department.expense_categories`
- `UploadBatch.transactions`
- `Transaction.upload_batch`
- `Transaction.expense_category`
- `Group.expense_category`
- `ExpenseCategory.groups`
- `ExpenseCategory.transactions`

backend/schema.sql

This file defines the PostgreSQL/Supabase schema for these operations.

Relevant tables:

- department
- users
- expense_category
- "group"
- upload_batch
- transaction

Relevant indexes:

- idx_expense_category_company
- idx_expense_category_department
- idx_group_expense_category
- idx_transaction_department_id
- idx_transaction_expense_category
- idx_transaction_batch_id
- idx_transaction_dedupe_hash

These indexes support department filtering, category filtering, batch lookup, and duplicate detection.

backend/app/database.py

This file configures database access:

- loads `.env`;
- reads `DATABASE_URL`;
- creates the SQLAlchemy engine;
- creates `SessionLocal`;
- provides `get_db` for FastAPI dependency injection;
- initializes tables and schema compatibility in `init_db`;
- seeds default expense categories.

For Supabase, `DATABASE_URL` points to the Supabase PostgreSQL connection string. No local PostgreSQL server is required by this code path.

`init_db` runs on application startup through `backend/app/main.py`.

Application Wiring

backend/app/main.py

This file creates the FastAPI app and includes the relevant routers:

- transaction_router
- grouping_router
- categorization_router

It also runs:

```
init_db()
```

on startup, then optionally seeds demo data if `AUTO_SEED_DEMO=True`.

backend/app/auth/router.py

The upload, grouping, and categorization endpoints depend on:

```
current_user: User = Depends(get_current_user)
```

`get_current_user` ensures these operations are authenticated. The operation-specific files do not manually parse JWTs; they rely on this dependency.

Endpoint Summary

Upload:

- POST /transactions/upload
- GET /transactions/dept/{dept_id}
- GET /transactions/{transaction_id}
- PATCH /transactions/{transaction_id}

Grouping:

- POST /grouping/assign-groups
- GET /grouping/dept/{dept_id}/groups
- GET /grouping/dept/{dept_id}/statistics

Transaction categorization:

- POST /categorization/predict

- `GET /categorization/dept/{dept_id}/summary`

Expense category workflow:

- `GET /categorization/expense-categories`
- `GET /categorization/dept/{dept_id}/expense-groups`
- `POST /categorization/expense/predict`
- `POST /categorization/expense-groups/approve`
- `POST /categorization/expense-groups/reject`

File-by-File Responsibilities

`frontend/pages/AdminDashboard.tsx`

Provides the user-facing controls for:

- uploading transaction files;
- running grouping;
- running transaction categorization;
- running expense-category suggestions;
- approving or rejecting suggested group categories;
- displaying grouped expense-category review data.

`frontend/lib/api.ts`

Centralizes frontend API calls. For these operations, it:

- sends multipart upload requests;
- sends grouping requests;
- sends categorization requests;
- sends expense suggestion, approval, and rejection requests;
- attaches auth headers through the shared `request` helper.

`frontend/types.ts`

Defines TypeScript shapes used by the frontend for:

- transactions;
- upload batches;
- expense categories;
- expense group summaries;
- categorization responses.

backend/app/main.py

Registers the upload, grouping, and categorization routers with FastAPI and initializes the database on startup.

backend/app/database.py

Creates database sessions, loads the `DATABASE_URL`, runs schema initialization, and seeds default expense categories.

backend/app/models.py

Defines the database objects used by these flows:

- Transaction
- UploadBatch
- Group
- ExpenseCategory
- Department
- User

backend/schema.sql

Defines the actual PostgreSQL/Supabase tables and indexes for transactions, upload batches, groups, and categories.

backend/app/transaction/router.py

Owns the upload endpoint and transaction read/update endpoints. It:

- receives uploaded files;
- validates department existence;
- parses files into dataframes;
- normalizes columns;
- resolves debit/credit fields;
- creates upload batch records;
- skips duplicates;
- inserts transaction rows;
- reports processed, failed, and duplicate row counts.

backend/app/services/dataframe.py

Owns dataframe-level file parsing and ledger normalization. It:

- reads CSV/XLS/XLSX bytes;
- normalizes common upload column aliases;
- converts debit/credit ledger exports into `amount` and `transaction_type`;
- can convert department transactions back into a dataframe for downstream analysis.

backend/app/services/common.py

Provides shared helpers for these flows. It:

- cleans chart account head text;
- computes upload duplicate hashes;
- checks required dataframe columns;
- normalizes boolean upload fields;
- defines valid simple transaction categories.

backend/app/grouping/router.py

Owns grouping endpoints. It:

- loads department transactions;
- computes transaction text;
- embeds transaction text;
- matches transactions to existing groups;
- creates new semantic groups;
- stores group assignment fields on transactions;
- returns grouping statistics.

backend/app/grouping/grouping_sbert.py

Owns embedding and similarity behavior. It:

- loads the SentenceTransformer model;
- embeds text with normalized vectors;
- provides a deterministic fallback embedding;
- computes cosine similarity.

backend/app/categorization/router.py

Owns both categorization workflows. It:

- assigns simple transaction labels;
- exposes categorization summaries;
- seeds and scopes expense categories;

- ensures groups exist for categorization;
- builds Gemini prompts;
- stores Gemini suggestions on groups;
- approves categories and applies them to group transactions;
- rejects pending suggestions.

backend/app/auth/router.py

Provides `get_current_user`, which protects the upload, grouping, and categorization endpoints.

Important Data Relationships

An uploaded file creates one `UploadBatch`.

Each inserted transaction points to:

- one `Department`;
- one optional `UploadBatch`;
- one optional `Group`, represented by `group_no` and `group_name`;
- one optional `ExpenseCategory`.

Each group belongs to:

- one department;
- one optional approved expense category.

Each expense category can be:

- global/system-level;
- company-level;
- department-level.

Operational Notes

- Uploading a file does not automatically run grouping or categorization.
- Grouping must be run explicitly from the dashboard.
- Expense-category suggestions must be explicitly requested.
- Gemini suggestions are not applied directly; they are stored as pending review.
- Admin approval is the step that writes `expense_category_id` onto both the group and its transactions.

- Duplicate prevention is department-scoped, so the same ledger row can exist in different departments but not be reinserted into the same department.
- The backend can use Supabase PostgreSQL through `DATABASE_URL`; these operations do not require a local PostgreSQL server.